Vérification Formelle de Logiciels

LOS

Chapitre 4

Détails du Chapitre 4

- Introduction
- Sémantique Opérationnelle
- Sémantique Axiomatique

Introduction

- Par vérification formelle d'un logiciel, on entend la vérification mathématique « exhaustive » de sa correction.
  - On cherche à prouver au sens mathématique que l'implantation (vue comme un modèle logique) satisfait à sa spécification (vue par exemple comme un ensemble de formules logiques).

- Notons que la vérification du logiciel peut être traitée sous les angles suivants :
  - Test : techniques et environnement de test pour le logiciel (voir chapitre 5)
  - Preuve : prouver qu'un logiciel satisfait à ses spécifications par la preuve de programme (objet de ce chapitre)
  - Model checking : modéliser le logiciel par des techniques à base d'automate et vérifier automatiquement des propriétés exprimées en logique temporelle (voir chapitre 6)

Introduction

- La vérification formelle est une activité complexe. Il faudrait d'abord procéder à l'extraction d'un modèle depuis un programme ou de formules logiques depuis des spécifications (souvent semi-formelles).

- La sémantique permet de répondre à ces questions et préparer le terrain pour la preuve des programmes et pour la correction des propriétés des logiciels.

Introduction

- La sémantique concerne l'aspect sens et signification à attribuer aux programmes :
  - Ce qu'ils calculent
  - Comment ils le calculent
  - Les objets mathématiques qu'ils représentent

- La sémantique permet, notamment, de :
  - Décider ce que doit faire un programme dans des cas complexes
  - Calculer si des programmes sont équivalents, ce qui permet à un compilateur de remplacer l'un par l'autre.
  - Dériver la plupart des analyses de programmes

Introduction

- La sémantique formelle (SF) concerne l'aspect dynamique (exécution) et elle comprend 3 approches :
  - Sémantique opérationnelle (SO)
  - Sémantique dénotationnelle (SD)
  - Sémantique axiomatique (SA)

Introduction

➔ **Sémantique Opérationnelle (SO)** : Vision impérative

- décrit le comportement exact de la machine lors de l'exécution du programme ;
- décrit l'évolution des états de la machine en cours d'exécution, ainsi que les transitions entre elles.
- Cette sémantique est orientée développeur de compilateur, ou interpréteur.

Remarque : Adaptée à la vérification formelle de propriétés sémantiques

Introduction

➔ **Sémantique Dénotationnelle (SD)** : vision fonctionnelle

- le programme est représenté par une fonction ; traduction des expressions du langage en un ensemble de dénotations sous forme de constantes ou de fonctions mathématiques.
- Elle s'intéresse à l'effet du programme et non à la manière avec laquelle il est exécuté.
- Elle est orientée concepteur de langages de programmation.

Remarque : Permet de définir des sémantiques plus abstraites

Introduction

➔ **Sémantique Axiomatique (SA)** : vision déclarative

- elle permet de prouver des propriétés des programmes sous forme d'assertions logiques.
- orientée programmeur pour vérifier la correction du programme.

Sémantique Opérationnelle

Sémantique Opérationnelle

→ Domaines et fonctions sémantiques :

Pour illustrer les diverses formes sémantiques, on considère un mini-langage impératif simple :

- **Catégories syntaxiques**

  NUM : l'ensemble des numéraux

  VAR : l'ensemble des variables (de noms)

  AEXP : l'ensemble des expressions arithmétiques

  BEXP : l'ensemble des expressions booléennes

  STM : l'ensemble des assertions (statements) (ensemble des commandes du langage)

- **Méta – variables**

  $n \in NUM\ ,\ x \in VAR$ x : variable du programme, emplacement mémoire.

  $a \in AEXP,\ b \in BEXP,\ s \in STM$

Sémantique Opérationnelle

Domaines et fonctions sémantiques :

Pour illustrer les diverses formes sémantiques, on considère un mini-langage impératif simple :

Syntaxe abstraite

Définir les catégories syntaxiques :

$n ::= 0|1|n0|n1$ (les numéraux sont définis dans le système binaire)

$a ::= n|x|a1 + a2|a1 * a2|a1 - a2$

$b ::= true|false|a1 = a2|a1 \leq a2|\neg(b)|b1 \wedge b2$

$s ::= x := a|skip|s1; s2|if\ b\ then\ S1\ else\ S2|while\ b\ do\ S$

Exemple :

$n = 110$

$n ::= n0 ::= n10 ::= 110$

Sémantique Opérationnelle

→ Sémantique des numéraux

- Domaine sémantique : l'ensemble des entiers $\mathbb{Z}$
- Fonction sémantique : $\mathcal{N}: NUM \longrightarrow \mathbb{Z}$

Exemple :

$\mathcal{N}[\![1]\!] = 1,\ \mathcal{N}[\![0]\!] = 0,\ \mathcal{N}[\![101]\!] = 5$

- Définition inductive de $\mathcal{N}$:

$$\mathcal{N}[\![1]\!] = 1, \qquad \mathcal{N}[\![0]\!] = 0$$

$$\mathcal{N}[\![n0]\!] = 2 * \mathcal{N}[\![n]\!], \qquad \mathcal{N}[\![n1]\!] = 2 * \mathcal{N}[\![n]\!] + 1$$

Exemple :

$$\mathcal{N}[\![101]\!] = 2 * \mathcal{N}[\![10]\!] + 1$$
$$= 2 * (2 * \mathcal{N}[\![10]\!]) + 1 \ = 2 * (2*1) + 1 = 5$$

Sémantique Opérationnelle

➔ Sémantique des expressions arithmétiques

L'interprétation des expressions arithmétiques dépend des valeurs associées aux variables présentent dans les expressions.

Un état est l'association d'une valeur (du bon type) à chaque variable. Si les variables sont toutes de type entier, un état s est une fonction définie par :

$$S : VAR \longrightarrow \mathbb{Z} \quad (\text{STATE} = \text{VAR} \longrightarrow \mathbb{Z})$$

Pour chaque variable x de VAR, l'état S spécifie une valeur pour x, notée Sx (ou S(x))

Par exemple, si Sx = 3 alors la valeur de x+1 dans S est 4

Remarque :

Autre notation : $[x \longmapsto 5, y \longmapsto 7, z \longmapsto 0]$: on veut spécifier que l'état est juste concerné par les variables x, y et z. Les autres variables sont indifférentes.

Sémantique Opérationnelle

→ Sémantique des expressions arithmétiques

Fonction sémantique : interprète les expressions arithmétiques

Elle est de la forme $\mathcal{A} : AEXP \longrightarrow (State \longrightarrow \mathbb{Z})$

Avec $State = VAR \longrightarrow \mathbb{Z}$

$$\mathcal{A}[\![n]\!]S = \mathcal{N}[\![n]\!]$$

$$\mathcal{A}[\![x]\!]S = Sx$$

$$\mathcal{A}[\![a1 + a2]\!]S = \mathcal{A}[\![a1]\!]S + \mathcal{A}[\![a2]\!]S \quad \text{(Idem pour – et *)}$$

La sémantique des expressions arithmétiques est définie inductivement sur la structure des exp. C'estune sémantique compositionnelle.

Exemple :

$$\mathcal{A}[\![x + 1]\!]S = \mathcal{A}[\![x]\!]S + \mathcal{A}[\![1]\!]S = Sx + \mathcal{N}[\![1]\!] = 3 + 1 = 4$$

Sémantique Opérationnelle

➔ Sémantique des expressions booléennes

La fonction sémantique est définie comme suit :

$$\mathcal{B} : BEXP \longrightarrow (State \longrightarrow \mathbb{B})$$

Avec $\mathbb{B} = \{tt, ff\}$

$$\mathcal{B}[\![True]\!]S = tt$$

$$\mathcal{B}[\![false]\!]S = ff$$

$$\mathcal{B}[\![a1 = a2]\!] = \begin{cases} tt \text{ si } \mathcal{A}[\![a1]\!]S = \mathcal{A}[\![a2]\!]S \\ ff \text{ sinon} \end{cases}$$

$$\mathcal{B}[\![a1 \leq a2]\!] = \begin{cases} tt \text{ si } \mathcal{A}[\![a1]\!]S \leq \mathcal{A}[\![a2]\!]S \\ ff \text{ sinon} \end{cases}$$

$$\mathcal{B}[\![non(b)]\!] = \begin{cases} tt \text{ si } \mathcal{B}[\![b]\!]S = ff \\ ff \text{ sinon} \end{cases}$$

$$\mathcal{B}[\![b1 \wedge b2]\!] = \begin{cases} tt \text{ si } \mathcal{B}[\![b1]\!]S = tt \text{ et } \mathcal{B}[\![b2]\!]S = tt \\ ff \text{ si } \mathcal{B}[\![b1]\!]S = ff \text{ ou } \mathcal{B}[\![b2]\!]S = ff \end{cases}$$

Exemple :

Si Sx=3

$$\mathcal{B}[\![\neg(x = 1)]\!]S = tt \text{ car } \mathcal{B}[\![x = 1]\!]S = ff$$

Sémantique Opérationnelle

➔ Sémantique des expressions booléennes

La fonction sémantique est définie comme suit :

$$\mathcal{B} : BEXP \longrightarrow (State \longrightarrow \mathbb{B})$$

Avec $\mathbb{B} = \{tt, ff\}$

$$\mathcal{B}[\![True]\!]S = tt$$

$$\mathcal{B}[\![false]\!]S = ff$$

$$\mathcal{B}[\![a1 = a2]\!] = \begin{cases} tt \text{ si } \mathcal{A}[\![a1]\!]S = \mathcal{A}[\![a2]\!]S \\ ff \text{ sinon} \end{cases}$$

$$\mathcal{B}[\![a1 \leq a2]\!] = \begin{cases} tt \text{ si } \mathcal{A}[\![a1]\!]S \leq \mathcal{A}[\![a2]\!]S \\ ff \text{ sinon} \end{cases}$$

$$\mathcal{B}[\![non(b)]\!] = \begin{cases} tt \text{ si } \mathcal{B}[\![b]\!]S = ff \\ ff \text{ sinon} \end{cases}$$

$$\mathcal{B}[\![b1 \wedge b2]\!] = \begin{cases} tt \text{ si } \mathcal{B}[\![b1]\!]S = tt \text{ et } \mathcal{B}[\![b2]\!]S = tt \\ ff \text{ si } \mathcal{B}[\![b1]\!]S = ff \text{ ou } \mathcal{B}[\![b2]\!]S = ff \end{cases}$$

Exemple :

Si Sx=3

$$\mathcal{B}[\![\neg(x = 1)]\!]S = tt \text{ car } \mathcal{B}[\![x = 1]\!]S = ff$$

Sémantique Opérationnelle

➔ Règles d'évaluation des expressions

On note par : $< a , S >$ l'évaluation de l'expression arithmétique a dans l'état S.

$< a , S > \longrightarrow n$: exprime le fait que l'expression a s'évalue en n, dans l'état S.

Règles : (prémisse au numérateur, conclusion au dénominateur)

$$(A1)\ \frac{}{<n ,S> \longrightarrow n}\ (n \in \mathbb{Z})$$

$$(A2)\ \frac{}{<x ,S> \longrightarrow Sx}\ (x \in VAR)$$

$$(A3)\ \frac{<a1 ,S> \longrightarrow n1 \quad <a2 ,S> \longrightarrow n2}{<a1+a2 ,S> \longrightarrow n1+n2}$$

$$(A4)\ \frac{<a1 ,S> \longrightarrow n1 \quad <a2 ,S> \longrightarrow n2}{<a1*a2 ,S> \longrightarrow n1*n2}$$

$$(A5)\ \frac{<a1 ,S> \longrightarrow n1 \quad <a2 ,S> \longrightarrow n2}{<a1-a2 ,S> \longrightarrow n1-n2}$$

Sémantique Opérationnelle

→ Règles d'évaluation des expressions

Exemple :

Supposons que Sx = 2

Prouver que $< (2 * 3) + x , S > \longrightarrow 8$

<u>But :</u> construire un arbre de dérivation en utilisant le système d'inférence (A1)-(A5) où l'expression cible est la racine.

$$(A3)\ \dfrac{(A4)\ \dfrac{(A1)\ \dfrac{}{< 2 , S > \longrightarrow 2} \quad (A1)\ \dfrac{}{< 3 , S > \ 3}}{< 2 * 3 , S > \longrightarrow 6} \quad (A2)\ \dfrac{}{< x , S > \longrightarrow 2}}{< (2 * 3) + x , S > \longrightarrow 8}$$

<(2 * 3) + x, s> → 8 est un théorème pour la sémantique donnée.

<3+2, s> → 1 n'est pas un théorème, il s'agit d'un jugement syntaxiquement correct mais sémantiquement erroné.

Sémantique Opérationnelle

→ Règles d'évaluation des expressions

Proposition 1 :

a1, a2 : deux expressions arithmétiques

$$a1 \sim a2 \iff (\forall v \in \mathbb{Z}, \forall s \in \{VAR \longrightarrow\}\mathbb{Z}\},$$

$$< a1, S > \longrightarrow v \iff < a2\,, S > \longrightarrow v)$$

Sémantique Opérationnelle

➔ Evaluation des commandes

Effet d'une expression : ne change pas l'état courant de la mémoire, elle l'inspecte.

(état avant) $S = [x \longmapsto 5] \Longrightarrow x + 1 \longrightarrow 6 \ \Longrightarrow [x \longmapsto 5]$ (état après) ===➔ Même état

Effet d'une instruction (cmd) : réalise des affectations, les valeurs des variables en mémoire sont modifiées.

(état avant) $\sigma = [x \longmapsto 5] \Longrightarrow x := x + 1 \Longrightarrow [x \longmapsto 6] = \sigma 2$ (état après) ====➔ Etat différent

Sémantique Opérationnelle

➔ Evaluation des commandes

Le rôle d'un programme (ou cmd) est de modifier l'état courant S1 de la mémoire en un état S2 :

$$< c, S1 > \longrightarrow S2$$

$< c, S1 >$ est appelée configuration

$< c, S1 > \longrightarrow S2$: transition (qui exécute la cmd c à partir de l'état S1, cette cmd se termine, et à la terminaison on se trouve dans l'état S2)

Le changement d'état, suite à une exécution de cmd, est noté :

$$S[x \longleftarrow m](y) = \begin{cases} m & si\ y = x \\ S(y) & sinon \end{cases}$$

Exprime l'état obtenu à partir de S en remplaçant x par m.

On peut écrire maintenant $< x := 5,\ S > \longrightarrow S[x \longleftarrow 5]$

Sémantique Opérationnelle

➔ Evaluation des commandes

Les règles de transition :

$$(C1)\ \frac{}{<skip, S> \longrightarrow S}$$ (Instruction vide)

$$(C2)\ \frac{<a, S> \longrightarrow n}{<x := a, S> \longrightarrow S[x \longleftarrow n]}$$ (Affectation)

$$(C3)\ \frac{<c1, S> \longrightarrow S1 \quad <c\ \ , S1> \longrightarrow S2}{<c1;c2, S> \longrightarrow S2}$$ (Séquencement)

$$(C4)\ \frac{<c1, S> \longrightarrow S1}{<i\ \ b\ then\ c1\ else\ c2\ , S> \longrightarrow S1}\ si\ \mathcal{B}[\![b]\!]S = tt$$

$$(C5)\ \frac{< c2,\ S > \longrightarrow S2}{< if\ b\ then\ c1\ else\ c2\ , S > \longrightarrow S2}\ si\ \mathcal{B}[\![b]\!]S = ff$$

(C4 à C5 représentent l'alternative)

Sémantique Opérationnelle

➔ Evaluation des commandes

Les règles de transition :

$$(C6)\ \frac{}{<while\ b\ do\ c\ ,S> \longrightarrow S}\ si\ \mathcal{B}[\![b]\!]S = ff$$

$$(C7)\ \frac{< c\ ,S > \longrightarrow S1 \quad < while\ b\ do\ c\ ,S1 > \longrightarrow S2}{< while\ b\ do\ c\ ,S > \longrightarrow S2}\ si\ \mathcal{B}[\![b]\!]S = tt$$

(C6 à C7 représentent la boucle)

Sémantique Opérationnelle

➔ Evaluation des commandes

Exemple :

Soit la cmd qui calcule le factoriel :

$$Fact = y := 1; while\ \neg(x = 1) do\ (y := y * x\ ; x := x - 1)$$

Si dans l'état initial S, on a Sx=3

Alors $< Fact, S > \longrightarrow S[y \longmapsto 6)[x \longmapsto 1]$ *(*)*

Pour montrer (*), il faut construire un arbre de dérivation à l'aide des règles (C1)-(C7) avec (*) comme racine.

Posons B= $y := y * x\ ; x := x - 1$

$$W = WHILE\ \neg(x = 1) do\ B$$

$$Fact = y := 1; W$$

Sémantique Opérationnelle

➔ Evaluation des commandes

$$\frac{< y := y * x , S[y \longrightarrow 3, x \longrightarrow 2] > \longrightarrow S[y \longrightarrow 6, x \longrightarrow 2] \quad < x := x - 1 , S[y \longrightarrow 6, x \longrightarrow 2] > \longrightarrow S[y \longrightarrow 6, x \longrightarrow 1]}{< B , S[y \longrightarrow 3, x \longrightarrow 2] > \longrightarrow S'}$$

$$\frac{< B , S[y \longrightarrow 3, x \longrightarrow 2] > \longrightarrow S[y \longrightarrow 6, x \longrightarrow 1] \quad < W , S' > \longrightarrow S'}{< W , S[y \longrightarrow 3, x \longrightarrow 2] > \longrightarrow S'}$$

$$\frac{< y := y * x , S[y \longrightarrow 1] > \longrightarrow S[y \longrightarrow 3] \quad < x := x - 1 , S[y \longrightarrow 3] > \longrightarrow S[y \longrightarrow 3, x \longrightarrow 2]}{< B , S[y \longrightarrow 1] > \longrightarrow S[y \longrightarrow 3, x \longrightarrow 2]}$$

$$\frac{< B , S[y \longrightarrow 1] > \longrightarrow S[y \longrightarrow 3, x \longrightarrow 2] \quad < W , S[y \longrightarrow 3, x \longrightarrow 2] > \longrightarrow S'}{< W , S[y \longrightarrow 1] > \longrightarrow S'}$$

$$(C3)\ \frac{< y := 1 , S > \longrightarrow S[y \longrightarrow 1] \quad < W , S[y \longrightarrow 1] > \longrightarrow S'}{< Fact , S > \longrightarrow S'}$$

Sémantique Axiomatique

Sémantique Axiomatique (SA)

- Par SA (ou sémantique à base d'état explicite), on considère les sémantiques dans lesquelles l'état d'un programme est décrit par des assertions logiques.

- La SA permet d'extraire des propriétés logiques des programmes à partir d'informations sur leurs états et sur l'évolution de l'état à la suite de l'exécution d'une instruction.

- La SA rend compte par le biais d'un assertion logique (AL), des changements causés par une instruction sur l'état d'un programme.

SO	SA
Manipule explicitement l'état du programme	AL qui exprime le changement suite à une évolution

- La SA se fonde sur l'association de sens logique aux constructeurs (du langage de programmation) : « assigning meaning to programs » dont les bases ont été définies par Hoare et Dijkstra.

Sémantique Axiomatique

➔ Logiques de Hoare et Dijkstra

La SA permet :

- D'associer des expressions logiques décrivant l'état du programme
- De définir par des propriétés logiques les changements apportés à un état à la suite de l'exécution d'une instruction.

On obtient ainsi un calcul (formel) sur les AL à partir des différentes opérations réalisées. Ce calcul est différent du calcul définit dans la SO. Il est destiné à la preuve des propriétés logiques sur les programmes.

Sémantique Axiomatique

➔ Logiques de Hoare et Dijkstra

Deux types de calcul (raisonnement) ont été définis :

- Un calcul descendant (raisonnement déductif). Les AL associées à un état sont transformées par l'exécution d'une instruction jusqu'à la fin du programme. Hoare adopte cette méthode.
- Un calcul ascendant (raisonnement abductif). En partant de la conclusion (à prouver), un calcul permet, à partir de l'instruction exécutée, de retrouver les prémisses (AL vraies). Dijkstra a suivi cette approche.

Sémantique Axiomatique

➔ Logique de Hoare

Tony Hoare

Proposée en 1969 par Tony Hoare.
(Hoare : prix Turing 1980)

La LH se base sur la définition de deux AL appelées Précondtion et Postccondition.

Une action (ou programme) S est complètement spécifiée par la donnée de ses deux AL :

$$\{P\}\ S\ \{Q\} \quad : \ \textit{triplet de Hoare}$$

P : précondition ; Q : postcondition

P indique les conditions logiques qui doivent être satisfaites avant l'exécution de S

Q indique, après l'exécution de S, les conditions logiques établies par S sachant que P est vérifié.

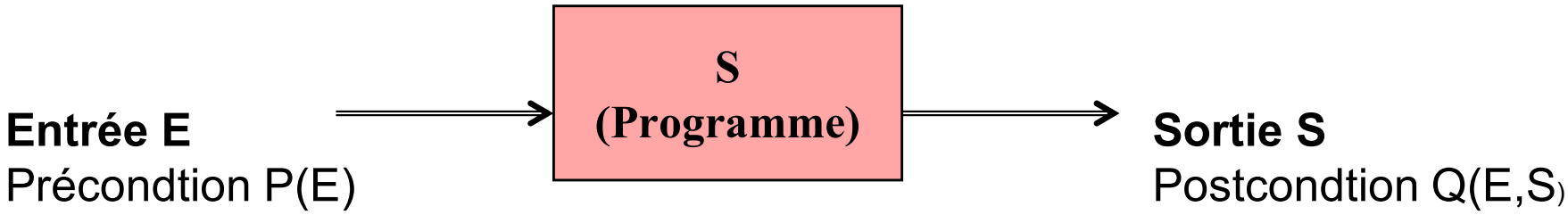
Sémantique Axiomatique

→ Logique de Hoare

Interprétation du triplet de Hoare :

$$\{P\}\ S\ \{Q\}$$

Si la propriété P est vraie pour les valeurs des variables du programme S avant son exécution et si l'exécution de S se termine alors la propriété Q est vraie après l'exécution de S.



Sémantique Axiomatique

Logique de Hoare

Exemple :

Programme Divide

E = {x, y}

```
a: = 0; b := x;
while (b >= y) do
    b :=: b –y;
    a := a + 1;
done
```

O = {a, b}

P(E) ⇔ y > 0 ∧ x > 0

Q(E, O) ⇔ x = ay + b ∧ b < y

Triplet de Hoare :

{y > 0 ∧ x > 0} Divide {x = a*y + b ∧ b < y}

Sémantique Axiomatique

Logique de Hoare

- Un triple de Hoare peut être vrai ou faux, exactement comme une formule logique
  - Triplet vrai : {x = 2} x := x+1 {x = 3}
  - Triplet faux : {x = 2} x := x+1 {x = 5}
- **Définition 1 : Plus faible condition**

  Soit un programme A et Q une postcondition.

  On dit que P est la plus faible précondition (wp : weakest pre-condition) qui conduit à Q en exécutant A si P est un prédicat qui définit le plus grand ensemble de valeurs des variables du programme A permettant toutes de passer dans un état qui satisfait Q, quand A est exécuté.
- **Exemple:**

  {x > 5} x := x + 1 {x > 6}, P = {x > 5} est la wp

  Par contre, {x > 6} n'est pas la wp (bien qu'elle conduit à x > 6)

Sémantique Axiomatique

Logique de Hoare

- **Définition 2 : (Plus forte condition)**

On dit que Q est la plus forte condition (sp : strongest condition) qui est atteinte en partant de P et en exécutant A, si Q est un prédicat définissant le plus petit ensemble de valeurs des variables du programme A qui sont atteintes à partir d'un état qui satisfait P, quand A est exécuté.

- **Exemple :**

$$\{x > 5\} \quad x := x + 1 \quad \{x > 6\}$$
$$\{x > 5\} \quad x := x + 1 \quad \{x > 3\}$$

sp : {x > 6}, par contre {x > 3} n'est pas une sp

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Correction des règles de Hoare : Etant donné un triplet t, s'il existe un arbre de déduction de Hoare complet ayant t à sa racine, alors le triplet est vrai. Les feuilles correspondent aux axiomes (règles sans prémisses).
- Construire un arbre de déduction nécessite la définition de la sémantique de chaque construction de base du langage de programmation :
  - Interprétation de SKIP : instruction neutre $\{p\}\ skip\ \{p\}$
  - Interprétation de l'affectation :

$$\{P[x \longleftarrow E]\}\ x := E\ \ \{P\} \qquad ou \qquad (Aff)\ \frac{}{\{P[x \longleftarrow E]\}\ x := E\ \ \{P\}}$$

L'axiome (Aff) se lit : pour que P soit vraie pour x, après l'exécution de x := E, il fallait qu'elle soit vraie pour E avant l'exécution.

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Interprétation de l'affectation :

$$\{P[x \leftarrow E]\}\ x := E\ \{P\} \qquad ou \qquad (Aff)\ \frac{}{\{P[x \leftarrow E]\}\ x := E\ \{P\}}$$

L'axiome (Aff) se lit : pour que P soit vraie pour x, après l'exécution de x := E, il fallait qu'elle soit vraie pour E avant l'exécution.

- Exemple

$$\textbf{(Aff)}\ \frac{}{\{x+1=3\}\ x := x+1\ \{x=3\}}$$

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Construire un arbre de déduction nécessite la définition de la sémantique de chaque construction de base du langage de programmation :
  - Interprétation de la séquence : la séquence ou la composition séquentielle est l'opération de base permettant de composer des programmes. Elle décrit l'enchaînement des instructions.

$$(Seq)\ \frac{\{P_1\}S\{P_2\}\ \ \{P_2\}S'\{P_3\}}{\{P_1\}\ S\ ;\ S'\{P_3\}}$$

  - **Exemple**
  - $$\frac{x\geq 0 \Rightarrow 0+x\geq 0 \quad (seq)\frac{\{0+x\geq 0\}\ a:=0\ \{a+x\geq 0\}\ ;\ \{a+x\geq 0\}\ b:=x\ \{a+b\geq 0\}}{\{0+x\geq 0\}a:=0\ ;b:=x\ \{a+b\geq 0\}} \quad a+b\geq 0\ \Rightarrow a\geq -b}{\{x\geq 0\}\ \ a:=0\ ;\ b:=x\ \ \{a\geq -b\}}$$

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Construire un arbre de déduction nécessite la définition de la sémantique de chaque construction de base du langage de programmation :
  - Interprétation de la conditionnelle

$$(cond)\ \frac{\{P \wedge C\}\, S\, \{Q\} \quad \{P \wedge non(C)\}\, S'\, \{Q\}}{\{P\}\ si\ C\ alors\ S\ sinon\ S' \{Q\}}$$

  - **Exemple**

$$\dfrac{\dfrac{\dfrac{}{\{X < 0\}\ X \leftarrow -X\ \{X > 0\}}}{\{(X \neq 0) \wedge (X < 0)\}\ X \leftarrow -X\ \{X > 0\}} \qquad \dfrac{\dfrac{}{\{X > 0\}\ \mathsf{skip}\ \{X > 0\}}}{\{(X \neq 0) \wedge (X \geq 0)\}\ \mathsf{skip}\ \{X > 0\}}}{\{X \neq 0\}\ \mathsf{if}\ X < 0\ \mathsf{then}\ X \leftarrow -X\ \mathsf{else\ skip}\ \{X > 0\}}$$

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Construire un arbre de déduction nécessite la définition de la sémantique de chaque construction de base du langage de programmation :
  - Interprétation de la répétition

$$\text{répétition} \begin{cases} \textit{Prédicat } I \textit{ désigant l'invariant}, I \textit{ est vrai, avant, pendant, et après la boucle} \\ \textit{entier naturel } V \textit{ représentant le variant qui converge vers la fin de la boucle} \end{cases}$$

$$\{\boldsymbol{I}\} \quad \textbf{while } \boldsymbol{C} \textbf{ do } \{\boldsymbol{I} \wedge \boldsymbol{C}\}\, \boldsymbol{S}\, \{\boldsymbol{I}\} \textbf{ end} \quad \{\boldsymbol{I} \wedge \neg(\boldsymbol{C})\}$$

$$(repeat) \quad \frac{\{I \wedge C\}\, S\, \{I\}}{\{I \textit{ tant que } C \textit{ faire } S \quad \{I \wedge \neg(C)\}} \qquad \textit{correction partielle}$$

$$(repeat) \quad \frac{\{I \wedge C \Longrightarrow V \in \mathbb{N}\} \qquad \{I \wedge C \wedge V = \gamma\}\, S\, \{I \wedge V < \gamma\}}{\{I \textit{ tant que } C \textit{ faire } S \quad \{I \wedge \neg(C)\}} \qquad \textit{correction totale}$$

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Construire un arbre de déduction nécessite la définition de la sémantique de chaque construction de base du langage de programmation :
  - Interprétation de la répétition

$$\{\boldsymbol{I}\} \quad \boldsymbol{while\ C\ do}\ \{\boldsymbol{I} \wedge \boldsymbol{C}\}\ \boldsymbol{S}\ \{\boldsymbol{I}\}\ \boldsymbol{end}\ \ \{\boldsymbol{I} \wedge \neg(\boldsymbol{C})\}$$

$$(repeat)\ \frac{\{I \wedge C \Longrightarrow V \in \mathbb{N}\} \qquad \{I \wedge C \wedge V = \gamma\}\ S\ \{I \wedge V < \gamma\}}{\{I\ tant\ que\ C\ faire\ S \quad \{I \wedge \neg(C)\}} \qquad correction\ totale$$

$\{I \wedge C \Longrightarrow V \in \mathbb{N}\}$: Typage

$\{I \wedge C \wedge V = \gamma\}\ S\ \{I \wedge V < \gamma\}$: Terminaison

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Règle de conséquence : Deux règles particulières sont souvent utilisées lors de la preuve de programme.
  - Renforcement de la précondition

$$(Renf)\ \frac{P' \Longrightarrow P \quad \{P\}\, S\, \{Q\}}{\{P'\}\, S\, \{Q\}}$$

  - Affaiblissement de la postcondition

$$(Affb)\ \frac{\{P\}\, S\, \{Q\} \quad Q \Longrightarrow Q'}{\{P\}\, S\, \{Q'\}}$$

  - Règle de conséquence

$$(cons)\ \frac{P \Longrightarrow P' \quad \{P'\}\, S\, \{Q'\} \quad Q' \Longrightarrow Q}{\{P\}\, S\, \{Q\}} \qquad : \ règle\ de\ conséquence$$

Sémantique Axiomatique

Sémantique axiomatique de Hoare

- Exemple :

- $$\frac{x \geq 0 \Rightarrow 0+x \geq 0 \quad (seq)\dfrac{\{0+x\geq 0\}a:=0\ \{a+x\geq 0\}\ ;\ \{a+x\geq 0\}\ b:=x\ \{a+b\geq 0\}}{\{0+x\geq 0\}a:=0\ ;b:=x\ \{a+b\geq 0\}} \quad a+b\geq 0\ \Rightarrow a\geq -b}{\{x\geq 0\}\ \ a:=0\ ;b:=x\ \ \{a\geq -b\}}$$

Sémantique Axiomatique

Logique de Dijkstra

- A et B deux assertions.

  $A \Longrightarrow B$: on dit que A est plus fort que B (et inversement, B est plus faible que A)

Plus une formule contient une précondition faible et une post condition forte, plus elle est informative et intéressante.

Sémantique Axiomatique

→ Logique de Dijkstra (LD)

Edsger Dijkstra

Proposé en 1975 par Edsger Dijkstra.
(Dijkstra : prix Turing 1972)

- La logique de Dijkstra (LD) a été développée après celle de Hoare.
- Dijkstra propose un calcul, sur les assertions, de type ascendant. Il propose donc, en partant d'une post condition, de trouver la précondition la plus faible, qui vérifie le triplet de Hoare.
- Ainsi, on raisonne toujours sur des triplets, mais les règles sont établies dans l'autre sens.

Sémantique Axiomatique

➔ LD : Concept de plus faible précondition

- Hypothèse :
  - S : un programme (commande ou spécification)
  - Q : une post condition (assertion logique)
- But :
  - Calculer la plus faible précondition P telle que {P} S {Q} est vérifié.
- Cette plus faible précondition est notée : $W_p(S,Q)$

  Par définition, on écrit : $\{P\}\ S\ \{Q\} \Leftrightarrow W_p(S,Q)$
- $W_p(S,Q)$: dénote la plus faible précondition telle que S termine et l'état satisfait Q après exécution de S.

Sémantique Axiomatique

➔ LD : Concept de plus faible précondition

- **Exemple :**

$W_p(x:=x+1, x>0) = x \geq 0$

$W_p(i:=i+1, i \leq n) = i<n$

$W_p(i:=0, i=0) = True$

$W_p(i:=0, i=1) = False$

La pfc P de $W_p(S,Q)$ est telle que :

$$\forall P' \left(P' \Longrightarrow W_p(S,Q)\right) \Longrightarrow (P' \Longrightarrow P)$$

Sémantique Axiomatique

→ LD : Calcul de la plus faible précondition (pfc)

Ci-dessous quelques règles de calcul de la pfc :

- Skip : $W_p(skip,Q) = Q$
- Affectation : $W_p(x:=e,Q) = Q[x:=e]$: remplacer toutes les occurrence de x par E

  ou $W_p(x := e, Q) = [x := e]\ Q$
- Séquence :

  $$W_p(S\ ;\ S'\ ,\ Q) = W_p(S\ ;\ W_p(S', Q))$$
- Conditionnelle :

$$W_p(\text{if } P \text{ then } S_1 \text{ else } S_2, Q) = \left(P \Longrightarrow W_p(S_1, Q)\right) \wedge \left(non(P) \Longrightarrow W_p(S_2, Q)\right)$$

Sémantique Axiomatique

➔ LD : Calcul de la plus faible précondition (pfc)

- Conditionnelle :

  W_p(if P then S_1 else S_2, Q) = $\left(P \Longrightarrow W_p(S_1, Q)\right) \wedge \left(non(P) \Longrightarrow W_p(S_2, Q)\right)$

- Exemple
  - W_p(if x > 0 then x := 1 else x := x+1, x = 1) = W_p(Ω, x = 1)
  - $\left(x > 0 \Longrightarrow W_p(x := 1, x = 1)\right) \wedge \left(non(x > 0) \Longrightarrow W_p(x := x + 1, x = 1)\right)$
  - $\left(non(x > 0) \vee W_p(x := 1, x = 1)\right) \wedge \left(non\left(non(x > 0)\right) \vee W_p(x := x + 1, x = 1)\right)$
  - $\left((x \leq 0) \vee W_p(x := 1, x = 1)\right) \wedge \left((x > 0) \vee W_p(x := x + 1, x = 1)\right)$
  - $\left((x \leq 0) \vee (1 = 1)\right) \wedge \left((x > 0) \vee (x = 0)\right)$
  - $\left((x \leq 0) \vee True\right) \wedge \left((x > 0) \vee (x = 0)\right) = True \wedge (x \geq 0) = (x \geq 0)$

W_p(Ω, x = 1) = $(x \geq 0)$

Sémantique Axiomatique

→ LD : Calcul de la plus faible précondition (pfc)

Ci-dessous Ci-dessous quelques règles de calcul de la pfc :

- Répétition : {P} T ; While C do S end {R}

  <= Condition suffisante :

  {P} T {I} -- établir l'invariant

  $(I \wedge C \implies W_p(S, I))$ -- Préservation de l'invariant

  $(I \wedge non(C) \implies R)$ -- sortie de la boucle

  $(I \wedge C \implies v \in \mathbb{N})$ -- variant

  $(I \wedge C \wedge v = \gamma \implies W_p(S, v < \gamma))$

  Remarque : $W_p(S, Q)$ = [S]Q

Sémantique Axiomatique

➔ LD : Calcul de la plus faible précondition (pfc)

Ci-dessous Ci-dessous quelques règles de calcul de la pfc :

- Répétition : {P} T ; While C do S end {R}
- **Exemple :**

  W_p(x:=0; while x<10 do x:=x+1 end, (x=10)) =

  INVARIANT : $0 \leq x < 10$

  VARIANT : 10 – x

  Remarque : $W_p(\Omega$, (x=10)) = [Ω] Q

Sémantique Axiomatique

→ LD : Calcul de la plus faible précondition (pfc)

- Exemple :

  W_p(x:=0; while x<10 do x:=x+1 end, (x=10))

  INVARIANT : $0 \leq x < 10$

  VARIANT : 10 – x

{P} T {I} -- établir l'invariant

[x := 0] I = [x := 0] $(0 \leq x < 10)$

$= 0 \leq 0 \leq 10$ → *True*

$(I \wedge C \implies W_p(S, I) = [S]I)$ -- Préservation de l'invariant

$$(0 \leq x \leq 10 \wedge x < 10) \implies [x := x + 1](0 \leq x \leq 10)$$

$$(x \leq 10 \wedge x < 10) \implies x + 1 \leq 10)$$

$(x \leq 10 \wedge x < 10) \implies x < 10)$ → *True*

Sémantique Axiomatique

→ LD : Calcul de la plus faible précondition (pfc)

- Exemple :

  W_p(x:=0; while x<10 do x:=x+1 end, (x=10))

  INVARIANT : $0 \leq x < 10$

  VARIANT : 10 – x

$(I \wedge non(C) \implies R)$ -- sortie de la boucle

$$(x \leq 10 \wedge non(x < 10)) \implies x = 10)$$

$$(x \leq 10 \wedge x \geq 10) \implies x = 10)$$ → *True*

$(I \wedge C \implies v \in \mathbb{N})$ -- variant

$$\left.\begin{array}{c}(x \leq 10 \wedge x < 10) \\ v \in \mathbb{N}\end{array}\right\} \implies 10 - x \in \mathbb{N}$$ → *True*

Sémantique Axiomatique

→ LD : Calcul de la plus faible précondition (pfc)

- Exemple :

  W_p(x:=0; while x<10 do x:=x+1 end, (x=10))

  INVARIANT : $0 \leq x < 10$

  VARIANT : 10 – x

$$(I \wedge C \ \wedge v = \ \gamma \Longrightarrow W_p(S, v < \gamma) = [S](v < \gamma))$$

$$(x \leq 10 \wedge x < 10 \wedge v = \gamma) \Longrightarrow (x \leq 10 \wedge x < 10 \wedge 10 \ - x = \gamma)$$

$$\Longrightarrow [x \ := x + 1\,]\,(v < \gamma)$$

$$\Longrightarrow [x \ := x + 1\,]\,(10 \ - x)$$

$$\Longrightarrow 10 - (x + 1) < \gamma \quad \rightarrow \textit{True}$$

Sémantique Axiomatique

→ **LD : Calcul de la plus faible précondition (pfc)**

- **Autres commandes :**
- W_p(PRE p then S end, Q) = P $\wedge W_p$(S,Q)
- W_p(Select P then S end,Q) = P $\Longrightarrow W_p$(S,Q)
- W_p(choice S or T end,Q) = W_p(S,Q) $\wedge W_p$(T,Q)

Sémantique Axiomatique

➔ LD : Calcul de la plus faible précondition (pfc)

- **Remarque :**

  W_p est une fonction à deux arguments : une instruction S et un prédicat Q.

  Pour un S fixé, on peut voir $W_p(S, Q)$ comme une fonction à un seul argument $W_{ps}(Q)$

  La fonction $W_{ps}(Q)$ est appelée transformateurs de prédicats. Il associe à Q la plus faible précondition P telle que {P} S {Q}.

Sémantique Axiomatique

➔ LD : Calcul de la plus faible précondition (pfc)

- **Exercice :**

$Wp(z:=2;\ y:=z+1;\ x:=y+z,\ x \in 3\ ..8)$

$= Wp(z:=2;\ Wp(y:=z+1;\ Wp(x:=y+z,\ x \in 3..8)))$

$= Wp(z:=2;\ Wp(y:=z+1, y+z \in 3..8))$

$= Wp(z:=2\ ;\ z+1+z \in 3..8)$

$= 2+1+2 \in 3..\ 8$

$= TRUE$